


# Team Project 1 - CAP

TypeScript Fundamental

Amjad

# Chapter 1
# Introduction to Typescript

Anjad

# What Is TypeScript?

TypeScript is a programming language developed by Microsoft that builds on JavaScript by adding a static type system and additional developer tooling.

- TypeScript is a superset of JavaScript, meaning valid JavaScript syntax is also valid TypeScript syntax.
- It introduces static type checking to help detect errors during development.
- It supports modern JavaScript features, including classes, modules, and asynchronous programming.
- TypeScript files typically use the .ts extension.
- TypeScript code can be compiled into JavaScript that runs in browsers and Node.js.

**JavaScript ( `app.js` )**

JavaScript

```
let username = "Ali";
let age = 22;
```

**TypeScript ( `app.ts` )**

TypeScript

```
let username: string = "Ali";
let age: number = 22;
```

**Key point:** TypeScript is JavaScript with an additional type system.

# JavaScript vs. TypeScript

Both languages use similar syntax, but they differ in how they handle types and detect errors.

| Feature | JavaScript | TypeScript |
|---|---|---|
| Typing | Dynamic | Static checking |
| Type annotations | No | Yes |
| Error detection | Mostly runtime | Many errors detected before runtime |
| Compilation | Not required | Usually compiled to JavaScript |
| File extension | .js | .ts |
| Browser execution | Directly supported | Typically requires JavaScript output |

JavaScript

```javascript
let price = 100;
price = "Free"; // Allowed
```

TypeScript

```typescript
let price: number = 100;
price = "Free"; // Type error
```

TypeScript detects that "Free" is a string and cannot be assigned to a variable declared as number.

Amjad

# Why Do We Need TypeScript?

JavaScript is flexible, but that flexibility can lead to unexpected behavior, especially in large applications.

**Common problems with JavaScript**

- Type-related errors: A function may receive an unexpected data type.
- Runtime errors: Some mistakes become visible only when the code executes.
- Code maintenance: Understanding variables and functions becomes more difficult as projects grow.
- Team collaboration: Developers may not know what types of values a function expects.

JavaScript

```javascript
function add(a, b) {
    return a + b;
}

console.log(add(10, 20));   // 30
console.log(add("10", 20)); // "1020"
```

TypeScript

```typescript
function add(a: number, b: number): number {
    return a + b;
}

console.log(add(10, 20));   // 30
console.log(add("10", 20)); // Type error
```

**Benefits**

TypeScript provides earlier error detection, improved code readability, better editor support, and more reliable refactoring.

# Advantages and Limitations of TypeScript

**Advantages**

- Static type checking catches many errors early.
- Better autocomplete and code suggestions in editors.
- Easier refactoring and code maintenance.
- Supports object-oriented programming features.
- Compatible with existing JavaScript libraries and frameworks.
- Improves collaboration through explicit type definitions.

**Limitations**

- Requires learning additional type-system concepts.
- Usually adds a compilation step to development.
- Type definitions can increase code complexity.
- Does not prevent all runtime errors.
- Type annotations are removed during compilation and do not automatically validate runtime data.

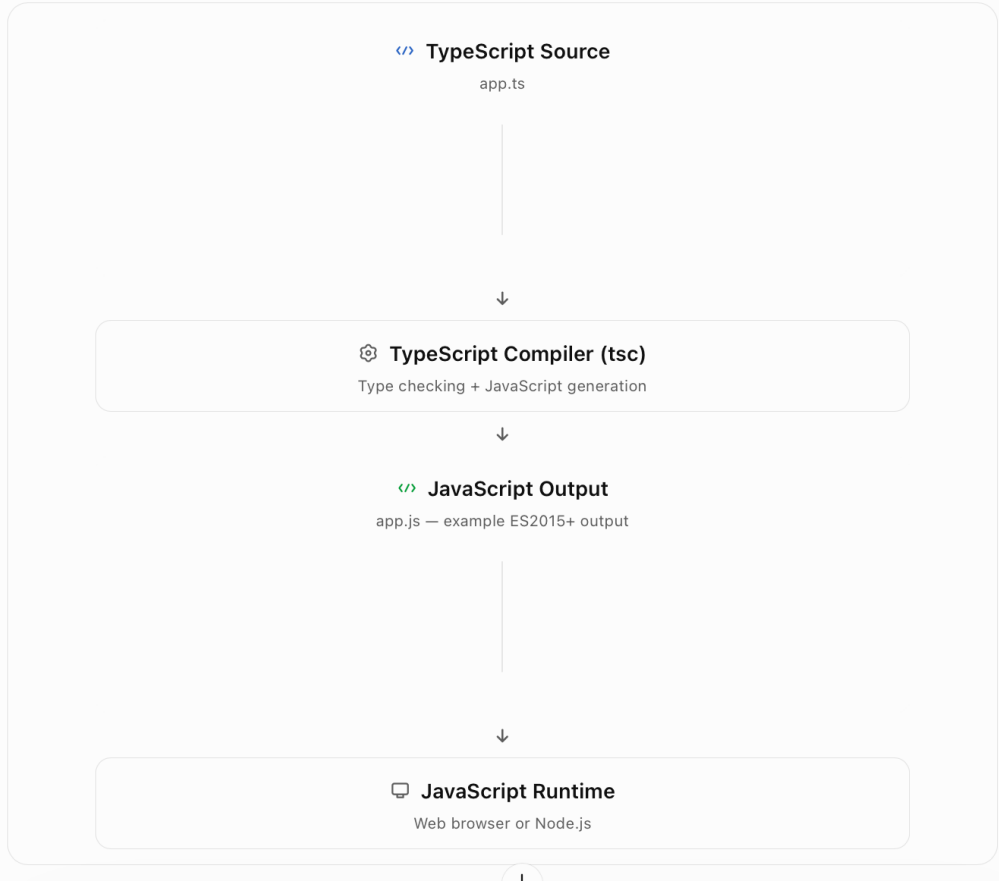
Amjad

# How TypeScript Works

Browsers generally execute JavaScript, **not** TypeScript source code directly.
TypeScript code is typically **compiled** (transpiled) into JavaScript using the TypeScript compiler (**tsc**).

**Important Concepts**

- The TypeScript compiler checks types and reports errors.
- Type annotations such as : number are removed from the generated JavaScript.
- The compiler can target different JavaScript versions.
- JavaScript output can be generated even when type errors exist, depending on compiler settings.
- TypeScript's static types do not exist as runtime checks in the generated JavaScript.




Amjad

# Chapter 2
# Installation & Setup

Anjad

# Prerequisites: Node.js, npm, and VS Code

Before writing TypeScript code, we need to prepare our development environment.

**1. Node.js**

A JavaScript runtime that allows us to execute JavaScript outside the browser.

- Used to run the JavaScript generated from TypeScript.
- Includes npm in standard installations.

Download Node.js

**2. npm (Node Package Manager)**

A package manager used to install and manage JavaScript and TypeScript packages.

- Installs the TypeScript compiler.
- Manages project dependencies.

Visual Studio Code - Download and install on Windows | Microsoft Store

**3. Visual Studio Code (VS Code)**

A code editor with built-in TypeScript language support.

- Syntax highlighting and autocomplete.
- Error detection and code suggestions.
- Integrated terminal.

Download VS Code

**Verify the Installation**

Open a terminal (Command Prompt, PowerShell, or VS Code terminal) and run:

```
$ node -v
$ npm -v
```

Anjad

# Installing TypeScript

TypeScript provides a compiler called tsc, which converts TypeScript code into JavaScript.

## Step 1: Create a Project Folder

```
$ mkdir typescript-tutorial
$ cd typescript-tutorial
```

## Step 2: Initialize the Project

```
$ npm init -y
```

This creates a package.json file to manage project information and dependencies.

## Step 3: Install TypeScript

```
$ npm install --save-dev typescript
```

This installs TypeScript as a development dependency of the project.

## Step 4: Verify the Installation

```
$ npx tsc --version
```

The command should display the installed TypeScript version.

**Note**: Installing TypeScript locally ensures that each project can use its own compiler version.

Amjad

# Creating Your TypeScript Program

**Step 1 — Create a File**

Open the project folder in VS Code and create a new file: app.ts

**Step 2 — Write TypeScript Code**

```
let studentName: string = "Ahmad";
let studentAge: number = 21;
let isEnrolled: boolean = true;

function greetStudent(name: string): string {
     return "Welcome, " + name + "!";
}

console.log(greetStudent(studentName));
console.log("Age:", studentAge);
console.log("Enrolled:", isEnrolled);
```

**Code Explanation**

- `string` specifies text values.
- `number` specifies numeric values.
- `boolean` specifies true or false.
- `name: string` defines a function parameter's type.
- `: string` after the parameter list specifies the function's return type.

Ahmad

# Compiling and Running TypeScript

TypeScript code must be converted into JavaScript for our standard Node.js execution workflow.

**Step 1: Compile the TypeScript File**
Open the terminal inside the project folder:

```
$ npx tsc app.ts
```
This generates a JavaScript file named app.js.

**Step 2 — Run the Generated JavaScript**

```
$ node app.js
```

**Important Notes**
- Browsers and standard JavaScript runtimes execute JavaScript.
- TypeScript type annotations are removed during compilation.
- The generated JavaScript file can be executed independently of the TypeScript compiler.
- If the source code changes, compile it again to update the JavaScript output.

**Note**: tsc compiles the code, while node executes the resulting JavaScript.

Amjad

# Understanding tsconfig.json

- The tsconfig.json file defines how the TypeScript compiler should check and compile our project.
- Instead of specifying individual files every time, we can configure the compiler once.

**Step 1: Create the Configuration File**

```
$ npx tsc --init
```

**Step 2: Configure the Project**

For our course, we can use the following simplified configuration:

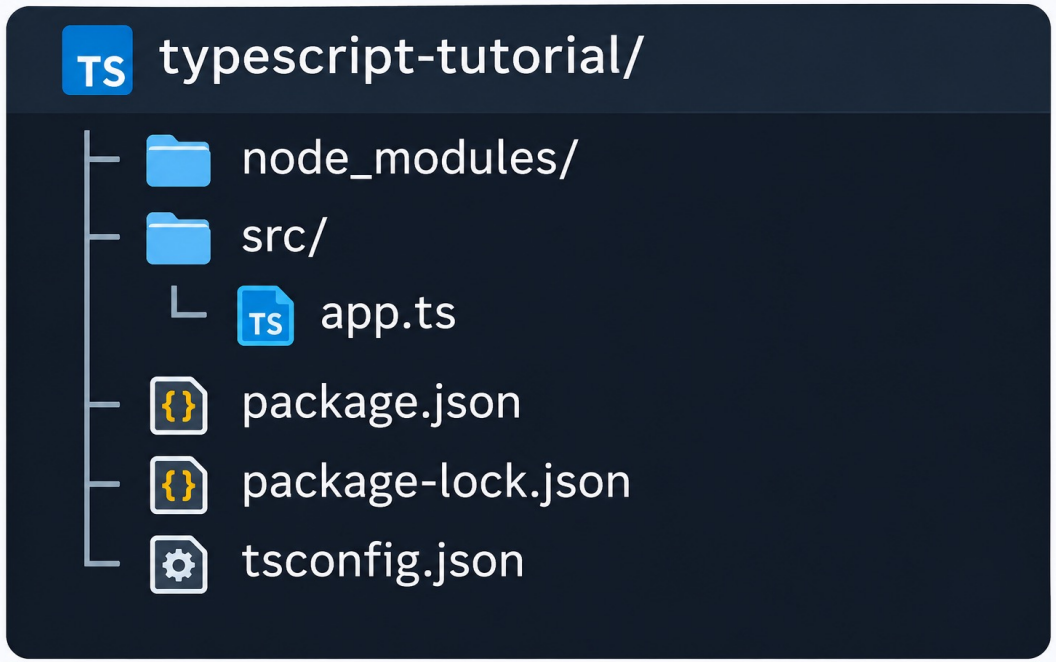
```
{
   "compilerOptions": {
      "target": "ES2020",
      "module": "CommonJS",
      "rootDir": "./src",
      "outDir": "./dist",
      "strict": true,
      "esModuleInterop": true
   },
   "include": ["src/**/*"]
}
```

| Option | Description |
|---|---|
| target | JavaScript language version to generate |
| module | Module format of generated JavaScript |
| rootDir | Root directory containing source files |
| outDir | Directory for generated JavaScript files |
| strict | Enables stricter type checking |
| esModuleInterop | Improves compatibility with CommonJS imports |
| include | Specifies which source files belong to the project |

Amjad

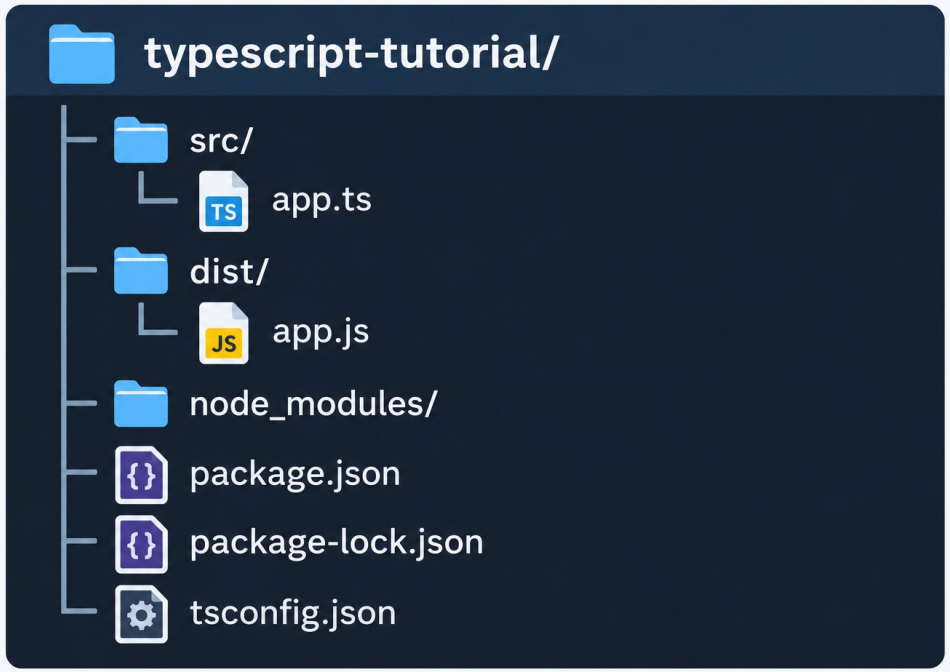
# Understanding tsconfig.json (Cont.)

**Step 3: Organize the Project**
Move app.ts into the src folder.




Step 4: Compile and Execute

```
$ npx tsc
$ node dist/app.js
```



# Chapter 3
# TypeScript Data Types

Arijad

# Variables and Type Annotations

In JavaScript, variables can hold values of different types. TypeScript allows us to explicitly specify the expected type of a variable using a type annotation.

**Syntax**

```
let variableName: type = value;
```

**Example**

```
let age: number = 20;

age = 25;        // Valid
age = "Twenty"; // Error: Type 'string'
                // is not assignable to type 'number'
```

Amjad

# Type Inference

- TypeScript does not always require us to explicitly declare the type of a variable.
- It can automatically determine the type based on the assigned value. This feature is called Type Inference.

```
//Explicit Type
let name: string = "Ali";
let age: number = 22;

//Inferred Type
let name = "Ali";
let age = 22;
```

In both cases, TypeScript understands that name is a string and age is a number.

**When Should We Use Explicit Types?**

- When defining function parameters.
- When declaring variables without initial values.
- When we want to make the expected type clear.
- When the intended type is broader than the inferred type.

**Note:** Type inference reduces unnecessary annotations while preserving type checking.

**Type Inference Error**

```
let grade = 95;

grade = 100;      // Valid
grade = "Excellent"; // Error
```

TypeScript infers grade as number from its initial value

# Primitive Data Types

TypeScript supports JavaScript's primitive data types and allows us to declare them explicitly.

| Type | Description | Example |
|---|---|---|
| string | Text values | "Hello" |
| number | Integer and decimal numbers | 25, 3.14 |
| boolean | True or false | true |
| null | Intentional absence of a value | null |
| undefined | Value not assigned | undefined |
| bigint | Large integers | 123n |
| symbol | Unique identifiers | Symbol("id") |

**Important Notes**

- TypeScript uses number for both integers and decimal values.
- Type names are written in lowercase: string, number, and boolean.
- With strictNullChecks enabled, null and undefined cannot be assigned to an ordinary string or numbervariable.

```
let studentName: string = "Ahmad";

// Error with strictNullChecks enabled
studentName = null;
```

**Enable strictNullChecks**

Open tsconfig.json and add:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "strictNullChecks": true
  }
}
```

Anjah

# Arrays and Tuples

**1. Arrays**

An array stores multiple values, usually of the same type.

**Syntax:**

```
let arrayName: type[] = [];
```

**Example:**

```
let grades: number[] = [85, 90, 95];
let students: string[] = ["Ali", "Sara", "Omar"];

grades.push(100);        // Valid
students.push("Ahmad"); // Valid

// grades.push("Excellent"); // Error
```

**2. Tuples**

A tuple is an array with a defined structure, where the types of elements at specific positions are known.

**Example:**

```
let student: [string, number, boolean];

student = ["Ahmad", 21, true]; // Valid

// Error: Incorrect element types
student = [21, "Ahmad", true];
```

# Arrays and Tuples (Cont.)

**Arrays vs. Tuples**

| Array | Tuple |
| --- | --- |
| Usually stores values of the same type | Can store values of different types |
| Length is generally flexible | Defines expected positions and length |
| Example: number[] | Example: [string, number] |

**Important:** Tuples are still JavaScript arrays at runtime. Some array operations, such as push(), are permitted by TypeScript even if they extend the tuple beyond its declared positions.

**Note**: Arrays define element types, while tuples describe the types and positions of elements.

Amjad

# The any and unknown Types

Sometimes we do not know the type of a value in advance, TypeScript provides any and unknown to handle such situations.

**1. The any Type**

The any type disables most type checking for that value.

```
let data: any = 10;

data = "Hello";
data = true;

console.log(data);
```

All these assignments are allowed. However, any can introduce runtime errors.

```
let value: any = "Hello";

value.toFixed(2); // No TypeScript error,
                  // but fails at runtime
```

**2. The unknown Type**

unknown can store any value, but TypeScript requires us to check its type before using type-specific operations.

```
let value: unknown = "Hello";

if (typeof value === "string") {
    console.log(value.toUpperCase());
}
```

**Note**: Prefer unknown over any when receiving values whose types are not yet known.

# The any and unknown Types (Cont.)

- The fundamental difference between any and unknown in TypeScript is **type safety**.
- While both types can hold **any arbitrary value**, any completely turns off TypeScript's type checking, whereas unknown forces you to safely verify the type before performing any operations on it.

simply: any means *"I don't care about the type,"* while unknown means *"I don't know the type yet, so check it first."*

```
let value: any = "Hello World";

// TypeScript allows all of these without a compiler error:
value.toUpperCase(); // Works fine at runtime
value.toFixed(2);    // Crashes at runtime! (Strings don't have .toFixed)
value();             // Crashes at runtime! (Not a function)
```

```
let value: unknown = "Hello World";

// TypeScript blocks these immediately at compile-time:
value.toUpperCase(); // Error: Object is of type 'unknown'
value.toFixed(2);    // Error: Object is of type 'unknown'

if (typeof value === "string") {
    // TypeScript now safely knows 'value' is a string inside this block
    console.log(value.toUpperCase()); // Works perfectly!
}
```

# Union Types

- A **Union Type** allows a variable to hold values of more than one specified type.
- We use the | symbol to combine types.

```
let studentId: number | string;

studentId = 12345;      // Valid
studentId = "ST12345"; // Valid

// studentId = true;    // Error
```

**Type Narrowing**

When using union types, we may need to check the actual type before performing certain operations.

```
function formatId(id: number | string): string {
    if (typeof id === "string") {
        return id.toUpperCase();
    }

    return id.toString();
}

console.log(formatId("st123"));
console.log(formatId(123));
```

# Literal Types and Type Aliases

TypeScript allows us to define types using specific values and to create reusable names for types.

**1. Literal Types**

A literal type restricts a variable to one or more exact values.

```
let status: "active" | "inactive";

status = "active";   // Valid
status = "inactive"; // Valid

// status = "pending"; // Error
```

This is useful when a variable should only accept predefined options.

**2. Type Aliases**

A Type Alias allows us to give a name to an existing or custom type using the type keyword.

```
type StudentId = number | string;

let id1: StudentId = 12345;
let id2: StudentId = "ST12345";
```

## Combining Literal Types with Type Aliases

```
type StudentStatus = "active" | "inactive" | "graduated";

let studentStatus: StudentStatus = "active";

studentStatus = "graduated"; // Valid
// studentStatus = "unknown"; // Error
```

# Chapter 4
# Functions and Objects

Amjad

# Typed Function Parameters

In JavaScript, function parameters can receive values of any type. TypeScript allows us to explicitly define the expected type of each parameter.

```
function calculateTotal(
    price: number,
    quantity: number
): number {
    return price * quantity;
}

console.log(calculateTotal(50, 3)); // 150
```

**Important Notes**

- Each parameter can have its own type.
- TypeScript checks the types of arguments passed to functions.
- Type annotations are removed when TypeScript is compiled into JavaScript.

**Note**: Typed parameters help prevent functions from receiving unexpected types of values.

Arjad

# Function Return Types

- A return type specifies the type of value a function is expected to return.
- In TypeScript, return types can be explicitly declared or **automatically inferred**.

```
function getStudentName(): string {
    return "Ahmad";
}

console.log(getStudentName());
```

**The void Return Type**

Use void when a function does not return a meaningful value.

```
function displayMessage(message: string): void {
    console.log(message);
}

displayMessage("Welcome to TypeScript!");
```

Amjad

# Optional and Default Parameters

TypeScript allows us to define function parameters that are not always required.
There are two common approaches: **optional parameters** and **default parameters**.

### 1. Optional Parameters

Use ? after a parameter name to make it optional.

```
function greetStudent(
    name: string,
    age?: number
): void {
    console.log("Name:", name);
    console.log("Age:", age);
}

greetStudent("Ahmad", 21);
greetStudent("Sara");
```

When an optional parameter is omitted, its value is undefined.

### 2. Default Parameters

A default parameter receives a predefined value when the caller does not provide one, or passes undefined.

```
function greet(
    name: string,
    message: string = "Welcome"
): string {
    return message + ", " + name;
}

console.log(greet("Ahmad"));
console.log(greet("Sara", "Hello"));
```

**Note**: Optional parameters provide flexibility, while default parameters provide fallback values.

# Arrow Functions in TypeScript

- Arrow functions provide a shorter syntax for writing functions.
- TypeScript allows us to add types to arrow function parameters and return values.

**Syntax**

```
const functionName = (parameter: type): returnType => {
    return value;
};
```

```
//Example
const add = ( a: number, b: number): number => {
    return a + b;
};
```

**Shorter Arrow Function**

When the function contains a single expression, we can omit the braces and return keyword.

```
const multiply = (a: number, b: number): number => a * b;

console.log(multiply(5, 4)); // 20

//Example: Arrow Function with an Array
const grades: number[] = [70, 85, 90, 95];

const passed = grades.filter(
    (grade: number): boolean => grade >= 80
);

console.log(passed); // [85, 90, 95]
```

Arijad

# Typed Objects

- In JavaScript, objects store data as key-value pairs.
- TypeScript allows us to define the expected structure of an object, including the names and types of its properties.

```
let student: {
    name: string;
    age: number;
    enrolled: boolean;
} = {
    name: "Ahmad",
    age: 21,
    enrolled: true
};
```

## Accessing and Updating Properties

```
console.log(student.name);

student.age = 22; // Valid

student.age = "Twenty Two"; // Error
```

## Missing or Incorrect Properties

```
let student: {
    name: string;
    age: number;
} = {
    name: "Sara"
    // Error: age is missing
};
```

## Optional Object Properties

Use ? to make an object property optional.

```
let student: {
    name: string;
    age?: number;
} = {
    name: "Sara"
};
```

# Interfaces

- An interface defines a contract describing the structure that an object must follow.
- Interfaces are especially useful when multiple objects share the same structure.

### Syntax

```
interface InterfaceName {
    propertyName: type;
}
```

### Example: Student Interface

```
interface Student {
    id: number;
    name: string;
    age: number;
    enrolled: boolean;
}
```

### Creating Objects Using an Interface

```
const student1: Student = {
    id: 1,
    name: "Ahmad",
    age: 21,
    enrolled: true
};

const student2: Student = {
    id: 2,
    name: "Sara",
    age: 22,
    enrolled: false
};
```

Both objects must satisfy the Student interface.

Amjad

# Interfaces (Cont.)

## Optional and Readonly Properties

```
interface Student {
    readonly id: number;
    name: string;
    age?: number;
}
```

- **readonly** prevents reassignment of a property through TypeScript's type system.
- **?** makes a property optional.

## Example: Interface with Functions

```
interface Student {
    id: number;
    name: string;
}

function printStudent(student: Student): void {
    console.log(student.id);
    console.log(student.name);
}

const student1: Student = {
    id: 1,
    name: "Ahmad"
};

printStudent(student1);
```

**Why Use Interfaces?**

- Avoid repeating object structures.
- Improve readability and maintainability.
- Help enforce consistent data structures.
- Support larger applications with many related objects.

**Note**: Interfaces define reusable contracts for the structure of objects.

# Type Aliases vs. Interfaces

- TypeScript provides two common ways to define reusable object types: type aliases and interfaces.
- Both can describe object structures, but they have different capabilities.

**Extending an Interface**

```typescript
interface Person {
    name: string;
}

interface Student extends Person {
    studentId: number;
}

const student: Student = {
    name: "Ahmad",
    studentId: 123
};
```

**Combining Type Aliases**

```typescript
type Person = {
    name: string;
};

type Student = Person & {
    studentId: number;
};

const student: Student = {
    name: "Ahmad",
    studentId: 123
};
```

# Type Aliases vs. Interfaces (Cont.)

```typescript
// Using an Interface
interface Student {
    id: number;
    name: string;
}

// Using a Type Alias
type Student = {
    id: number;
    name: string;
};

// Both definitions can be used in the same way:
const student: Student = {
    id: 1,
    name: "Ahmad"
};
```

| Feature | Interface | Type Alias |
|---|---|---|
| Define object structures | Yes | Yes |
| Define primitive aliases | No | Yes |
| Define union types | No | Yes |
| Extend object structures | extends | Intersection & |
| Declaration merging | Yes | No |
| Implemented by classes | Yes | Yes, for compatible object types |

**When Should We Use Each?**

- Use interfaces when defining object structures, especially those intended to be extended or implemented by classes.
- Use type aliases for unions, tuples, primitives, and other complex type combinations.
- For ordinary object types, either approach is valid.

Amjad

# Chapter 5
# Object-Oriented TypeScript

Amjad

# Introduction to Classes

- TypeScript supports **Object-Oriented Programming (OOP)**, which allows us to organize code using classes and objects.
- A **class** is a blueprint used to create objects. It defines the properties (data) and methods (behavior) that objects can have.
- An **object** is an instance of a class.

## Syntax

```
class ClassName {
    propertyName: type;

    constructor(value: type) {
        this.propertyName = value;
    }

    methodName(): void {
        // Method body
    }
}
```

## Example: Student Class

```
class Student {
    name: string;
    age: number;

    constructor(name: string, age: number) {
        this.name = name;
        this.age = age;
    }

    displayInfo(): void {
        console.log(this.name, this.age);
    }
}

const student1 = new Student("Ahmad", 21);
student1.displayInfo();
```

## Important Concepts

- **`class`** defines a new class.
- **`new`** creates an object from a class.
- **`this`** refers to the current object.
- Properties store data, while methods define behavior.
- Each object can have its own property values.

Amira

# Properties and Constructors

- A class can contain properties, which represent its data, and a constructor, which initializes objects when they are created.
- The constructor is automatically called when we create an object using new.

```
class Student {
    id: number;
    name: string;
    grade: number;

    constructor(id: number, name: string, grade: number) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }

    hasPassed(): boolean {
        return this.grade >= 50;
    }
}

const student = new Student(1, "Sara", 85);

console.log(student.name); // Sara
console.log(student.hasPassed()); // true
```

# Properties and Constructors (Cont.)

**Constructor Parameter Properties**

TypeScript also provides a shorter way to declare and initialize properties directly in the constructor.

```
class Student {
    constructor(
        public id: number,
        public name: string,
        public grade: number
    ) {}

    hasPassed(): boolean {
        return this.grade >= 50;
    }
}

const student = new Student(2, "Omar", 45);

console.log(student.name);         // Omar
console.log(student.hasPassed()); // false
```

# Access Modifiers: public, private, protected

- Access modifiers control where class properties and methods can be accessed.
- TypeScript provides three main access modifiers.

| Modifier | Inside class | Subclass | Outside class |
|---|---|---|---|
| public | Yes | Yes | Yes |
| private | Yes | No | No |
| protected | Yes | Yes | No |

Amjad

# Access Modifiers: public, private, protected

### Example: public and private

```
class BankAccount {
    public owner: string;
    private balance: number;

    constructor(owner: string, balance: number) {
        this.owner = owner;
        this.balance = balance;
    }

    getBalance(): number {
        return this.balance;
    }

    deposit(amount: number): void {
        if (amount > 0) {
            this.balance += amount;
        }
    }
}

const account = new BankAccount("Ahmad", 1000);

console.log(account.owner);       // Ahmad
console.log(account.getBalance()); // 1000

account.deposit(500);
console.log(account.getBalance()); // 1500

// account.balance = 5000; // TypeScript error
```

### Example: protected

```
class Person {
    protected name: string;

    constructor(name: string) {
        this.name = name;
    }
}

class Student extends Person {
    displayName(): void {
        console.log(this.name);
    }
}

const student = new Student("Sara");
student.displayName(); // Sara

// console.log(student.name); // Error
```

**Note**: Access modifiers support encapsulation by controlling access to class members.

# Readonly Properties

- The readonly modifier prevents a class property from being reassigned after initialization.
- Readonly properties can be initialized in their declaration or inside the constructor.

```
class Student {
    readonly id: number;
    name: string;

    constructor(id: number, name: string) {
        this.id = id;
        this.name = name;
    }
}

const student = new Student(101, "Ahmad");

student.name = "Mohammad"; // Valid

// student.id = 202; // TypeScript error
```

**readonly vs. const**

| const | readonly |
|---|---|
| Used for variable bindings | Used for object and class properties |
| Prevents variable reassignment | Prevents property reassignment through TypeScript |
| Can reference an object | Can be used inside classes and interfaces |

# Inheritance

- **Inheritance** allows one class to reuse and extend the properties and methods of another class.
- The original class is called the **parent class** (or base class), while the new class is called the **child class** (or derived class).
- TypeScript uses the **extends** keyword to implement inheritance.

```
class Person {
    constructor(public name: string) {}

    introduce(): void {
        console.log("My name is " + this.name);
    }
}

class Student extends Person {
    constructor(
        name: string,
        public studentId: number
    ) {
        super(name);
    }

    study(): void {
        console.log(this.name + " is studying.");
    }
}

const student = new Student("Ahmad", 123);

student.introduce();
student.study();
```



# Inheritance (Cont.)

**Method Overriding**

A child class can provide its own implementation of an inherited method.

```
class Person {
    introduce(): void {
        console.log("I am a person.");
    }
}

class Student extends Person {
    override introduce(): void {
        console.log("I am a student.");
    }
}

const student = new Student();
student.introduce(); // I am a student.
```

**Important Notes**

- extends creates an inheritance relationship.
- super() calls the parent class constructor.
- A child class can introduce additional properties and methods.
- override explicitly identifies a method that replaces an inherited implementation.

**Note**: Inheritance promotes code reuse and allows classes to specialize existing behavior.

# Implementing Interfaces

- An interface defines a contract that a class can follow.
- A class uses the implements keyword to indicate that it satisfies an interface.

**Syntax**

```
interface InterfaceName {
  methodName(): void;
}

class ClassName implements InterfaceName {
  methodName(): void {
    // Implementation
  }
}
```

```
interface Printable {
    printInfo(): void;
}

class Student implements Printable {
    constructor(
        public name: string,
        public age: number
    ) {}

    printInfo(): void {
        console.log(this.name, this.age);
    }
}

const student = new Student("Sara", 22);
student.printInfo();
```

Amir

# Implementing Interfaces (Cont.)

### What Happens if a Method Is Missing?

```
interface Printable {
    printInfo(): void;
}

// Error: Missing printInfo() implementation
class Student implements Printable {
    name: string = "Ahmad";
}
```

### Implementing Multiple Interfaces

```
interface Printable {
    printInfo(): void;
}

interface Identifiable {
    getId(): number;
}

class Student implements Printable, Identifiable {
    constructor(
        public id: number,
        public name: string
    ) {}

    printInfo(): void {
        console.log(this.name);
    }

    getId(): number {
        return this.id;
    }
}
```

Anjah

# Chapter 6
# TypeScript in Real Applications

Amjad

# TypeScript and the DOM

- The Document Object Model (DOM) represents an HTML document as a collection of objects that JavaScript and TypeScript can access and modify.
- TypeScript uses the same DOM APIs as JavaScript, but adds type checking and editor support.

**Example:**

```
const title = document.getElementById("title");
const message = document.getElementById("message");

if (title !== null) {
    title.textContent = "Welcome to TypeScript!";
}

if (message !== null) {
    message.textContent = "Learning DOM manipulation.";
}
```

**Why Check for null?**

The getElementById() method returns:

- An HTMLElement when the element exists.
- null when no element with that ID exists.

With **strictNullChecks** enabled, TypeScript requires us to handle this possibility.

**Type Assertions with HTML Elements**

Sometimes we need to tell TypeScript the specific type of an HTML element.

```
const input = document.getElementById("studentName") as HTMLInputElement | null;

if (input !== null) {
    console.log(input.value);
}
```

**HTMLInputElement** provides access to input-specific properties such as value.

**Note**: TypeScript makes DOM manipulation safer by checking element types and potential null values.

# Working with HTML Elements and Events

- TypeScript supports JavaScript event handling while allowing us to specify and check event-related types.
- Events occur when users interact with a webpage, such as clicking buttons, typing text, or submitting forms.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>TypeScript Events</title>
</head>
<body>
    <input type="text" id="studentName"
           placeholder="Enter your name">

    <button id="greetBtn">Greet Student</button>

    <p id="result"></p>

    <script src="app.js"></script>
</body>
</html>
```

```typescript
const nameInput = document.getElementById(
    "studentName"
) as HTMLInputElement | null;

const greetButton = document.getElementById("greetBtn");
const result = document.getElementById("result");

if (nameInput && greetButton && result) {
    greetButton.addEventListener("click", (): void => {
        const name: string = nameInput.value;

        result.textContent = "Hello, " + name + "!";
    });
}
```

- addEventListener() attaches an event handler to an element.
- TypeScript can infer event types from known event names and element types.
- Type assertions should only be used when the expected element type is known.
- DOM events are handled in the browser, not in a standard Node.js environment without DOM support.

TypeScript allows us to handle browser events while checking the types of elements and event objects.

# Modules: import and export

- As applications grow, keeping all code in one file becomes difficult.
- Modules allow us to organize code into separate files and reuse functions, classes, and variables.
- TypeScript supports JavaScript's import and export syntax.

## Named export:

File: math.ts

```ts
export function add(a: number, b: number): number {
    return a + b;
}

export function multiply(a: number, b: number): number {
    return a * b;
}
```

File: app.ts

```ts
import { add, multiply } from "./math";

console.log(add(10, 20));
console.log(multiply(5, 4));
```

## Default export:

```ts
export default function greet(): string {
    return "Hello!";
}
```

```ts
import greet from "./greet";
```

# Modules: import and export (Cont.)

- A file can have multiple named exports.
- A module can have one default export.
- Modules help organize applications into reusable components.
- TypeScript supports different module formats, including CommonJS and ES Modules.

| Feature | Default Export | Named Export |
|---|---|---|
| **Quantity per file** | Strictly one | Unlimited |
| **Export Syntax** | export default function greet() {} | export function greet() {} |
| **Import Syntax** | import greet from "./greet"; | import { greet } from "./greet"; |
| **Curly Braces {}** | No braces | Requires braces |
| **Naming on Import** | Can be renamed freely without extra syntax. | Must match the exact export name (unless using as). |

Ahmad

# Author


## Amjad Ataallah

Email: amjad.atallah@najah.edu
Linkedin: https://www.linkedin.com/in/ataallah/
Website: https://amjadataallah.github.io

## Yanal Sweseh

Email: yanal.swaseh@najah.edu
Linkedin: https://www.linkedin.com/in/yanalsweseh/
Website: https://github.com/yswesieh